

PRODUCT BACKEND ENGINEER TEST

Concert Ticket Booking Platform

Backend System Design & Implementation Report

Engineering Focus

Correctness under concurrency

Idempotent retries •

Voucher consistency

Reservation lifecycle • Operational workflows

Testable engineering trade-offs

Version: 1.0

Submission date: August 23, 2026

Package: Report, source code, and diagram files

Prepared for the Product Backend Engineer technical assessment.

Contents

1	Executive Summary	1
2	Requirements and Engineering Risks	3
2.1	Business Context	3
2.2	Actors	3
2.3	Functional Requirements	4
2.4	Non-Functional Requirements	4
2.5	Use Case Diagram	5
2.6	Engineering Risks and Design Goal	5
3	Scope, Assumptions, Business Rules, and Invariants	7
3.1	Implemented Scope	7
3.2	Assumptions	8
3.3	Business Rules	8
3.4	Critical Invariants	9
3.5	Non-Goals	9
4	System Architecture	10
4.1	Architecture Goals	10
4.2	System Context	10
4.3	Selected Architecture: Modular Monolith and PostgreSQL	11
4.4	Component and Module Architecture	11
4.5	Why PostgreSQL Is the Consistency Boundary	12
4.6	Alternatives and Trade-offs	13
4.7	Evolution Triggers	13
5	Database Design	14
5.1	Data Model Goals	14
5.2	Core Entities	14
5.3	Entity Relationship Diagram	14
5.4	Critical Constraints	16
5.5	Index Strategy	16
5.6	Price Snapshotting	16

6	Booking Workflow and Transaction Boundary	17
6.1	Create Booking Sequence	17
6.2	Transaction Boundary	18
6.3	Failure Cases	18
6.4	Multi-Category Reservation	18
7	Concurrency and Overselling Prevention	19
7.1	Why Check-Then-Update Is Unsafe	19
7.2	Atomic Conditional Inventory Update	19
7.3	Alternatives Considered	20
7.4	Concurrency Strategy	20
7.5	Correctness Evidence	21
8	Idempotency and Retry Safety	22
8.1	Failure Scenario	22
8.2	API Contract	22
8.3	Database Enforcement	22
8.4	Request Fingerprint	23
8.5	Concurrent Retry Evidence	23
9	Voucher and Promotion Consistency	24
9.1	Voucher Validation	24
9.2	Atomic Global Quota Update	24
9.3	Per-User Anti-Abuse Rule	24
9.4	Release Semantics	25
9.5	Operation Scope	25
9.6	Correctness Evidence	25
10	Booking Lifecycle and Reservation Expiry	26
10.1	State Model	26
10.2	Why Reservation TTL Exists	27
10.3	Expiry Worker	27
10.4	Scalable Worker Claiming	28
10.5	Confirm-vs-Expire Race	29
11	API and Operation Workflows	30
11.1	Representative Customer APIs	30
11.2	Representative Operation APIs	31
11.3	Availability Semantics	31
11.4	Error Contract	31
12	Security, Reliability, and Observability	32
12.1	Security Boundary	32
12.2	Reliability Controls	32

12.3 Observability	33
12.4 Implementation Status and Evolution	33
13 Testing Strategy and Correctness Evidence	34
13.1 Testing Layers	34
13.2 Invariant-to-Test Evidence	35
13.3 Requirement Traceability	35
13.4 Evidence Standard	36
14 Performance and Flash-Sale Analysis	37
14.1 Expected Workload and Evaluation Goal	37
14.2 Executed Environment	37
14.3 Executed Scenarios	38
14.4 Measured Results	38
14.5 Interpretation and Limitations	38
15 Scope, Limitations, and Future Evolution	39
15.1 Implemented and Omitted Scope	39
15.2 Known Limitations	39
15.3 Payment Processing Scope	40
15.4 Decision Summary and Revisit Triggers	40
16 Conclusion	41

Executive Summary

The Concert Ticket Booking Platform serves customer reservation flows and internal operation workflows. Its principal risks are not raw CRUD complexity but contention over finite ticket inventory and voucher quota, duplicate bookings caused by network retries, invalid lifecycle transitions, and burst traffic during flash sales.

The submitted solution is a **modular monolith** exposing a stateless REST API on Node.js 22 and NestJS, with PostgreSQL 16 as the source of truth and consistency boundary. Booking, inventory, voucher, idempotency, operation, and expiry-worker responsibilities remain logically separated while sharing one local ACID transaction where correctness requires coordination. Docker Compose, versioned SQL migrations, deterministic seed data, OpenAPI, and Postman assets make the solution reproducible.

The critical booking path uses conditional inventory updates, deterministic resource ordering, database-backed idempotency with request fingerprints, atomic voucher quota reservation, server-side price calculation, immutable price snapshots, and a ten-minute **RESERVED** TTL. A polling worker claims expired rows with **FOR UPDATE SKIP LOCKED**; confirmation, cancellation, manual operation changes, and expiry all use conditional lifecycle writes so only one competing terminal transition can win.

Table 1.1: Final correctness and performance evidence

Evidence	Verified result
Automated tests	12 unit suites / 32 tests; 7 integration suites / 22 tests; 4 concurrency suites / 7 tests
Oversell race	100 attempts for 10 tickets produce exactly 10 bookings and final inventory zero
Retry and voucher races	One logical booking per idempotency key and at most one winner for the final voucher unit
Lifecycle race	Confirm versus expiry produces exactly one valid terminal state with no double release
k6 baseline	501 requests, 501 booking successes, 0.000% system errors, p95 24.87 ms and p99 31.73 ms
Delivery gates	Format, lint, typecheck, build, Newman smoke, submission and delivery verification pass in CI

The scope intentionally excludes real payment processing, seat-level selection, refunds, notifications, full catalog/voucher CRUD, and multi-region deployment. Confirmation is simulated or performed through an authorized current workflow. A future external payment provider would be called only after the reservation transaction commits; verified callbacks would drive an idempotent **RESERVED** -> **CONFIRMED** transition. These boundaries keep the assessment focused on demonstrable booking correctness without presenting future infrastructure as already implemented.

Requirements and Engineering Risks

2.1 Business Context

Customers browse published concerts, reserve finite category inventory, apply eligible vouchers, and track a deliberately small booking lifecycle. Operators and administrators investigate bookings and perform guarded maintenance. The assessment workload is approximately 300–500 booking requests per minute; the hard problem is synchronized demand for the same last tickets or voucher units.

2.2 Actors

Table 2.1: Current-scope actors

Actor	Primary goals	Trust boundary
Customer	Browse, reserve, retry safely, view, confirm, and cancel own bookings	Seeded <code>x-user-id</code> ; assessment identity, not production authentication
Operator	Monitor/investigate bookings and execute permitted workflows	Requires <code>OPERATOR</code> or <code>ADMIN</code> ; manual changes are audited
Administrator	Current elevated operation capabilities	Separate business actor sharing the submitted role policy
System Worker	Expire overdue reservations and release resources	Internal background process; no public endpoint

Payment is not a current actor because real processing is out of scope; the boundary is summarized in Chapter 15.

2.3 Functional Requirements

“Explicit” means stated by the brief; “Derived” supports the stated user workflow; “Engineering” preserves correctness and operability.

Table 2.2: Functional requirement summary

ID	Requirement	Actor	Source	Status
FR-01	Browse published concerts	Customer	Explicit	Implemented
FR-02	View concert details	Customer	Derived	Implemented
FR-03	View categories and server prices	Customer	Explicit	Implemented
FR-04	View informational availability	Customer	Derived	Implemented
FR-05	Create a time-limited reservation	Customer	Explicit	Implemented
FR-06	Apply an eligible voucher	Customer	Explicit	Implemented
FR-07	View owned booking detail/status	Customer	Derived	Implemented
FR-08	View own booking history	Customer	Derived	Implemented
FR-09	Monitor and filter bookings	Operator / Admin.	Explicit	Implemented
FR-10	Inspect booking and audit detail	Operator / Admin.	Derived	Implemented
FR-11	Perform guarded manual transition with reason	Operator / Admin.	Explicit	Implemented
FR-12	Create draft concerts and categories	Operator / Admin.	Explicit	Implemented
FR-13	Publish a valid future concert with inventory	Operator / Admin.	Derived	Implemented
FR-14	Create a validated scoped voucher	Operator / Admin.	Explicit	Implemented
FR-15	Update/delete concert or voucher definitions	Operator / Admin.	Explicit	Not implemented
FR-16	Confirm/cancel a live owned reservation	Customer	Derived	Implemented
FR-17	Expire overdue reservations automatically	Worker	Engineering	Implemented
FR-18	Release inventory after cancel/expiry	Worker	Engineering	Implemented
FR-19	Release reserved voucher quota once	Worker	Engineering	Implemented
FR-20	Replay repeated booking intent idempotently	Customer/System	Engineering	Implemented

2.4 Non-Functional Requirements

Table 2.3: High-value non-functional requirements

ID	Requirement	Acceptance evidence	Status
NFR-01	Inventory remains non-negative under contention	DB checks + CONC-001	Met
NFR-02	One logical request reserves resources once	Unique key/fingerprint CONC-002	+ Met
NFR-03	Voucher global/per-user limits remain consistent	Conditional update/index CONC-003	+ Met
NFR-04	Booking creation is atomic	Rollback integration tests	Met
NFR-05	Flash-sale failures are controlled	Stable 4xx, limiter, k6 metrics	Met
NFR-06	Support 300–500 booking requests/minute locally	501-request k6 run	Shown locally
NFR-07	Lifecycle races produce one valid terminal state	Conditional writes + CONC-004	Met
NFR-08	Enforce ownership, roles, validation, and safe config	API/integration/security checks	Partial
NFR-09	Maintain testable module boundaries	Modular monolith + automated suites	Met

ID	Requirement	Acceptance evidence	Status
NFR-10	Reproduce and observe the system locally	Compose, migrations, seed, Met probes, logs, metrics	

2.5 Use Case Diagram

Figure 2.1 contains actor goals only. Atomic reservation, idempotency, and transition enforcement belong to later design diagrams. The submission also includes the Mermaid sources under `docs/diagrams/` for independent zooming and review.

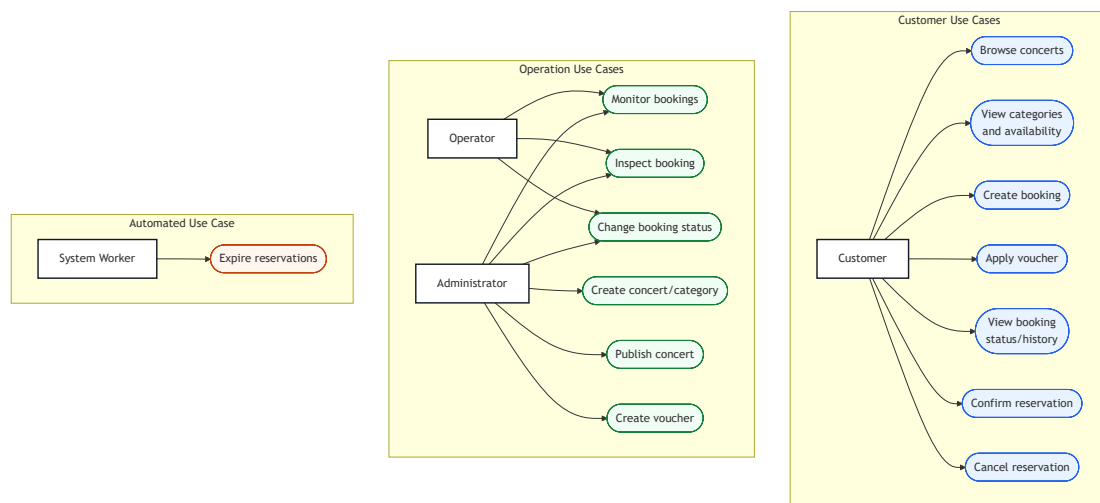


Figure 2.1: Current-scope actor goals

2.6 Engineering Risks and Design Goal

Table 2.4: Primary risk-to-design mapping

Risk	Failure mode	Engineering response
Overselling	Inventory sold beyond capacity	Atomic conditional PostgreSQL update
Duplicate retry	Multiple bookings from one intent	User-scoped idempotency key, fingerprint, and replay
Voucher abuse	Global/per-user quota exceeded	Atomic quota reservation and redemption constraint
Lifecycle race	Conflicting terminal state or double release	Conditional transition from RESERVED
Flash-sale instability	Hot rows and retry storms	Short transactions, controlled errors, limiter, and evidence

The design deliberately prioritizes correctness under concurrent and retried booking requests over architectural breadth.

Scope, Assumptions, Business Rules, and Invariants

3.1 Implemented Scope

The submission covers concert browse/detail, category inventory, transactional booking creation, voucher application, customer booking lookup, confirm/cancel, operator monitoring and guarded changes, minimal concert/category/voucher creation, and automated expiry. Real authentication, payment, seat maps, notifications, and multi-region operation are excluded.

3.2 Assumptions

Table 3.1: Assumptions that shape the design

ID	Assumption
A-01	PostgreSQL 16 is the authoritative store and consistency boundary.
A-02	The submitted <code>x-user-id</code> identifies seeded users; it is not production authentication.
A-03	Inventory is category-level capacity, not assigned seating.
A-04	Prices and discounts are calculated from persisted server data; money uses fixed precision.
A-05	A new booking is <code>RESERVED</code> for ten minutes unless confirmed, cancelled, or expired.
A-06	One idempotency key represents one normalized request for one customer.
A-07	Voucher reservation consumes quota while the booking is active and can be released once.
A-08	API instances are stateless; correctness must survive restarts and multiple instances.

3.3 Business Rules

These rules express user/product policy rather than concurrency guarantees:

- requested quantity is positive and each category belongs to the concert;
- only a published, future concert can be booked;
- voucher status, time window, scope, and per-user policy must match;
- the server owns price, discount, and total calculations;
- a live reservation may be confirmed or cancelled only before expiry;
- operation changes require an elevated role, a valid transition, and an audit reason.

3.4 Critical Invariants

Table 3.2: Invariant, enforcement, and evidence

ID	Invariant	Main enforcement	Evidence
INV-01	Inventory never becomes negative	Conditional update + DB checks	CONC-001
INV-02	One logical request consumes stock once	Idempotency constraint/fingerprint	CONC-002
INV-03	Voucher quota never exceeds its limit	Conditional quota + unique redemption	CONC-003
INV-04	Booking creation is all-or-nothing	One PostgreSQL transaction	INT-001
INV-05	Cancellation/expiry releases once	Conditional lifecycle transition	CONC-004
INV-06	Historical booking prices do not change	Item and discount snapshots	INT-002
INV-07	Only one valid terminal transition wins	Update requires RESERVED	CONC-004
INV-08	Manual/worker changes are traceable	Status history in same transaction	Operation tests

3.5 Non-Goals

The assessment does not implement production identity, real payments/refunds, seat selection, notification delivery, full administration CRUD, analytics, multi-region active-active data, or event-broker infrastructure. These boundaries keep the evidence focused on finite-resource correctness.

System Architecture

4.1 Architecture Goals

The architecture must keep correctness-sensitive transactions short and understandable, preserve strong consistency for finite resources, support the stated workload without speculative infrastructure, remain maintainable and testable, reproduce locally, and expose explicit evolution seams.

4.2 System Context

Figure 4.1 shows only the submitted system. PostgreSQL is the sole external consistency service; payment and notification providers are kept out of the current architecture.

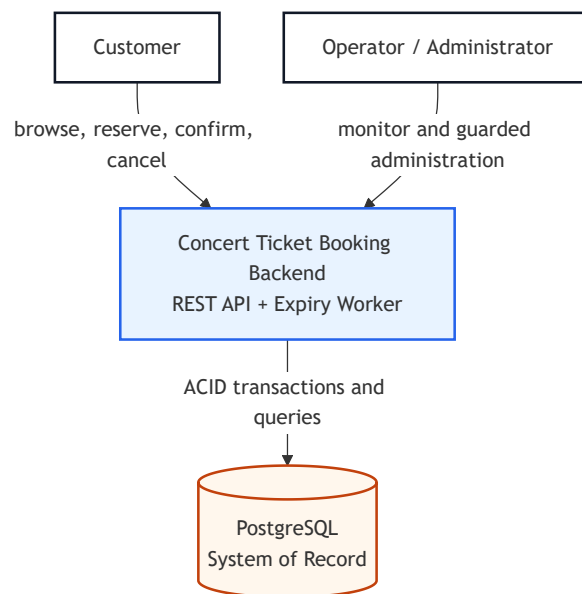


Figure 4.1: Current-scope system context

4.3 Selected Architecture: Modular Monolith and PostgreSQL

Decision Use a modular monolith for the assessment

Booking, inventory, voucher, and idempotency require strong transactional coordination. Keeping these modules in one deployable process and one database avoids distributed transactions while retaining explicit domain boundaries. The stateless API can still be replicated horizontally when measured demand requires it.

This selection is proportional to the stated 5–8.3 booking requests per second. Microservices would add network failure modes, service discovery, cross-service retry semantics, and distributed coordination without improving the finite-resource guarantees demonstrated by this submission.

4.4 Component and Module Architecture

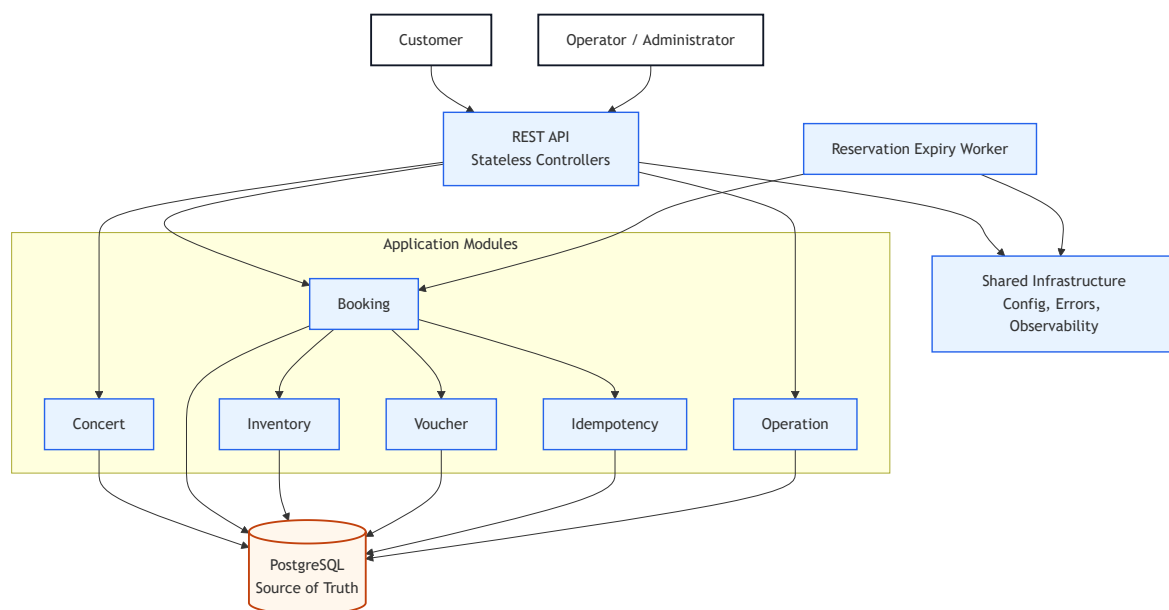


Figure 4.2: Modular-monolith component view

Table 4.1: Logical modules and responsibilities

Module	Responsibility
Concert	Published catalog, categories, prices, and informational availability
Inventory	Atomic ticket reservation and exactly-once release primitives
Booking	Aggregate creation/read, server pricing, transaction orchestration, and lifecycle
Voucher	Eligibility, scope, discount policy, quota reservation/consume/release
Idempotency	Request fingerprint, user/key claim, conflict, and response replay
Operation	Elevated booking queries, guarded manual transitions, concert/category/voucher creation
Expiry Worker	Batch claim of overdue reservations and transactional resource release
Shared	Configuration, database/transaction helpers, validation, errors, rate limit, and observability

The dependency direction is **Controller** -> **Application/Use Case** -> **Domain and Repository** ports -> **Infrastructure**. Controllers translate HTTP concerns; they do not own business transactions.

4.5 Why PostgreSQL Is the Consistency Boundary

Ticket inventory, voucher quota, booking state, audit history, and idempotency records are relational and participate in the same business outcomes. PostgreSQL conditional writes, row locks, constraints, and ACID rollback make it the natural arbiter. Redis may later support a shared limiter/cache, but it is not authoritative inventory and no Redis lock is needed for correctness.

4.6 Alternatives and Trade-offs

Table 4.2: Architecture alternatives considered

Decision point	Alternative	Reason not selected now
Deployment boundary	Microservices	Cross-service consistency and operational overhead exceed the assessment need
Inventory coordination	Redis distributed lock	Adds a second failure/authority system around data already owned by PostgreSQL
Critical workflow	Eventual event-driven reservation	Cannot provide the same simple all-or-nothing booking result without larger saga semantics
Expiry processing	Broker-delivered delayed messages	Reliable broker scheduling is useful at scale but unnecessary for the current bounded polling workload

4.7 Evolution Triggers

Scale stateless API replicas after measuring database contention. Multiple expiry workers already coordinate through `FOR UPDATE SKIP LOCKED`. A transactional outbox becomes relevant only when a real post-commit consumer exists; hot-row admission control or partitioning requires measured lock-wait evidence. Chapter 15 summarizes revisit triggers.

Database Design

5.1 Data Model Goals

The schema must make finite-resource correctness enforceable in PostgreSQL, support short transactions, retain historical monetary values, provide an auditable booking lifecycle, and serve ownership/operation/expiry queries with purpose-built indexes.

5.2 Core Entities

Concert/category/inventory tables own the sellable catalog; booking/items own the reservation and price snapshot. Voucher redemptions, idempotency records, and status history preserve promotion, retry, and lifecycle evidence. The ERD is the compact source for table relationships and critical fields.

5.3 Entity Relationship Diagram

Figure 5.1 shows the implemented schema. A transactional `outbox_events` table is intentionally absent because the outbox remains an ADR-backed future option rather than submitted runtime behavior.

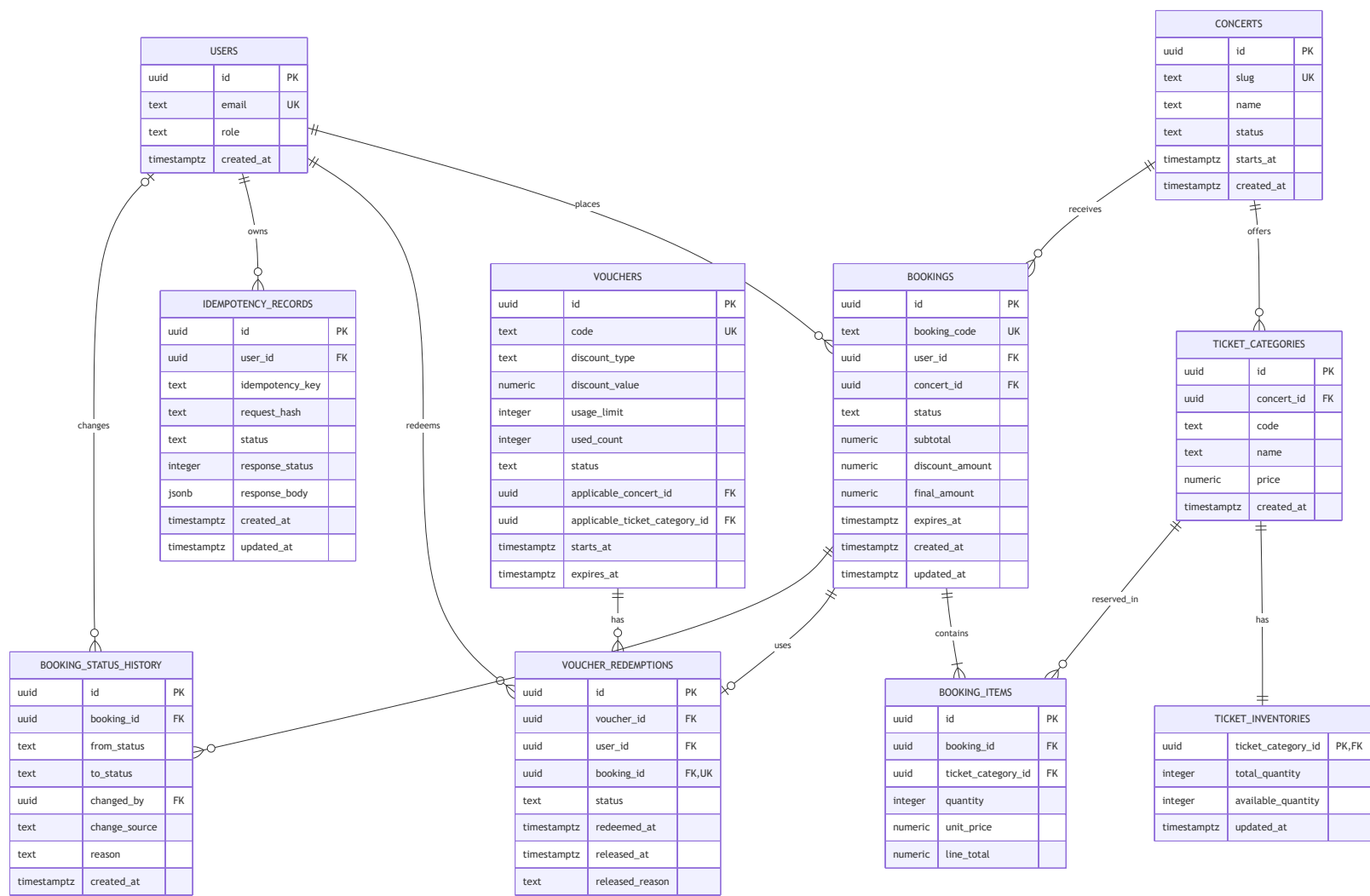


Figure 5.1: Implemented PostgreSQL entity relationship model

5.4 Critical Constraints

```

1 ALTER TABLE ticket_inventories
2   ADD CONSTRAINT chk_inventory_non_negative
3   CHECK (available_quantity >= 0),
4   ADD CONSTRAINT chk_inventory_not_above_total
5   CHECK (available_quantity <= total_quantity);
6
7 CREATE UNIQUE INDEX uq_idempotency_user_key
8   ON idempotency_records(user_id, idempotency_key);
9
10 CREATE UNIQUE INDEX uq_voucher_user
11   ON voucher_redemptions(voucher_id, user_id)
12   WHERE status <> 'RELEASED';

```

Listing 5.1: Representative database constraints

5.5 Index Strategy

Indexes should follow real query patterns rather than blanket indexing.

Table 5.1: Example query-to-index mapping

Query Pattern	Suggested Index
Expire old reservations	bookings(status, expires_at)
Customer booking history	bookings(user_id, created_at)
Operation filter by concert/status	bookings(concert_id, status, created_at)
Voucher lookup by code	unique vouchers(code)
Idempotency replay	unique idempotency_records(user_id, idempotency_key)

5.6 Price Snapshotting

The request never supplies an authoritative total. The server reads current ticket price and promotion policy, calculates the amount, and persists immutable snapshots in `booking_items` and `bookings`. This prevents future catalog changes from rewriting booking history.

Booking Workflow and Transaction Boundary

6.1 Create Booking Sequence

The create-booking use case converts one authenticated customer intent into one replayable result. The full vector sequence diagram is supplied separately at `docs/figures/booking-sequence.pdf`, with editable source at `docs/diagrams/booking-sequence.mmd`. It is not embedded here because its replay, conflict, sold-out, voucher, and rollback branches become too small at normal report zoom. The same flow is captured compactly below.

The implemented sequence is:

1. authenticate the assessment identity, validate the DTO, normalize items, and calculate a canonical request fingerprint;
2. inspect an already-completed idempotency record for an immediate replay/conflict path;
3. for a new logical request, begin one database transaction and claim the user, key, and fingerprint;
4. validate that the concert and categories are bookable;
5. reserve every category by atomic conditional update in deterministic order;
6. validate and reserve voucher policy/quota when supplied;
7. calculate subtotal, discount, and final amount from persisted server data;
8. insert the booking, immutable items, initial status history, and completed idempotency response;
9. commit and return the result.

6.2 Transaction Boundary

The idempotency claim/completion, ticket reservation, optional voucher quota and redemption, price snapshots, booking/items, and initial audit history share one transaction. A failure in any dependent operation rolls all of them back. This avoids a completed idempotency record for a booking that never committed and avoids inventory or voucher leakage after later validation fails.

External calls such as email, analytics, or a future payment provider must not extend this database transaction. They occur only after commit or through a reliable post-commit pattern such as the optional transactional outbox.

6.3 Failure Cases

Table 6.1: Create-booking failure semantics

Failure	API behavior	Database outcome
Invalid/unpublished concert or category	Controlled validation/business error	No resource or booking mutation
Insufficient inventory	INSUFFICIENT_TICKET_INVENTORY conflict	Whole request rolls back; stock remains valid
Invalid/ineligible/exhausted voucher	Stable voucher error	Earlier inventory writes roll back
Same key, same fingerprint	Original status/body replayed	No second booking or decrement
Same key, changed fingerprint	IDEMPOTENCY_KEY_REUSED conflict	Existing logical result remains untouched
Unexpected database error	Sanitized server error	Transaction rollback; no partial success is stored

6.4 Multi-Category Reservation

Category identifiers are normalized and processed in deterministic order. The consistent lock/update order reduces deadlock risk, while one transaction makes the request all-or-nothing: failure to reserve any category automatically rolls back every earlier category and optional voucher write.

Concurrency and Overselling Prevention

7.1 Why Check-Then-Update Is Unsafe

A sequence that first reads `available_quantity` and later updates it can oversell because concurrent transactions may observe the same pre-update value.

7.2 Atomic Conditional Inventory Update

```
1 UPDATE ticket_inventories
2 SET available_quantity = available_quantity - :quantity
3 WHERE ticket_category_id = :ticket_category_id
4    AND available_quantity >= :quantity
5 RETURNING available_quantity;
```

Listing 7.1: Atomic inventory reservation

A returned row means the reservation succeeded. No returned row means there was insufficient inventory. The correctness decision is made by PostgreSQL rather than by a stale application read.

7.3 Alternatives Considered

Table 7.1: Inventory concurrency alternatives

Option	Correctness potential	po-	Assessment trade-off
Application read then update	Unsafe		Race window exists between the read and write
Pessimistic row lock	Correct		Explicit lock/read/write path is longer than the selected one-statement predicate
Optimistic version column	Correct with retry		Adds version/retry policy for a resource naturally guarded by quantity predicate
Atomic conditional update	Correct; selected		PostgreSQL decides against current row state in one statement
Redis distributed lock	Potentially correct with careful failure handling		Creates a second coordination authority and lock-expiry failure modes
Queue serialization	Correct but asynchronous	asyn-	Adds queue latency/operations and changes synchronous request semantics

7.4 Concurrency Strategy

Table 7.2: Concurrency hazards and controls

Hazard	Control
Two customers compete for final ticket	Atomic conditional update
Booking contains multiple categories	Deterministic category order + one DB transaction
Voucher and inventory have different outcomes	Same transaction; any failure rolls back both
Admin/worker changes stale booking state	Conditional status update based on current state
Multiple expiry workers	Row claim with <code>FOR UPDATE SKIP LOCKED</code> (optional plus point)

7.5 Correctness Evidence

CONC-001 seeds 10 tickets and launches 100 simultaneous booking attempts. The verified result is exactly 10 successful bookings, 90 controlled conflicts, final inventory zero, and no negative intermediate/final state. This checks persisted bookings and inventory rather than relying only on response codes.

Idempotency and Retry Safety

8.1 Failure Scenario

A booking can be committed successfully while the HTTP response is lost. The client then retries because it cannot know whether the original request succeeded. Without idempotency, the retry may reserve inventory again and create a second booking.

8.2 API Contract

```
1 POST /bookings HTTP/1.1
2 Idempotency-Key: 44a3b9d0-...
3 Content-Type: application/json
4
5 {
6   "concertId": 1,
7   "items": [
8     { "ticketCategoryId": 10, "quantity": 2 }
9   ],
10  "voucherCode": "FLASH20"
11 }
```

Listing 8.1: Idempotent booking request

8.3 Database Enforcement

The idempotency key is scoped by authenticated user and protected by a unique constraint. The record stores a canonical request fingerprint and enough result data to identify/replay the completed logical booking.

8.4 Request Fingerprint

The canonical request can include user ID, concert ID, sorted ticket items, quantities, and normalized voucher code. A cryptographic hash (for example, SHA-256) is stored with the key.

Table 8.1: Idempotency behavior

Situation	Behavior
New key	Process the booking and persist logical result
Same key + same fingerprint	Return the original logical booking/result
Same key + different fingerprint	Reject with an idempotency conflict
Concurrent requests with same key	Exactly one request claims the logical operation; others replay/wait/reload according to implementation policy

8.5 Concurrent Retry Evidence

CONC-002 sends 20 concurrent requests with the same customer, idempotency key, and payload. The final database state contains one logical booking and one inventory decrement; every completed retry resolves to that same result. A companion integration path verifies that reusing the key with a different fingerprint returns a stable conflict and performs no new mutation.

Voucher and Promotion Consistency

9.1 Voucher Validation

Voucher validation includes status, validity window, optional concert/category scope, remaining global quota, and one-use-per-user policy. A global voucher has NULL scope columns; a scoped voucher must match the booking concert and at least one requested category. The server calculates the discount; the client cannot submit an authoritative discount amount.

9.2 Atomic Global Quota Update

```
1 UPDATE vouchers
2 SET used_count = used_count + 1
3 WHERE id = :voucher_id
4     AND status = 'ACTIVE'
5     AND starts_at <= NOW()
6     AND expires_at > NOW()
7     AND used_count < usage_limit
8 RETURNING used_count;
```

Listing 9.1: Atomic voucher quota reservation

The conditional update and redemption insert occur inside the same transaction as inventory and booking persistence. A voucher rejection therefore cannot leave a ticket reservation behind, and a later booking failure cannot consume promotion quota.

9.3 Per-User Anti-Abuse Rule

For a one-use-per-user policy, a partial unique index on (`voucher_id`, `user_id`) where redemption status is not `RELEASED` provides a database-backed guarantee. A retry of

the same logical booking does not create another redemption because idempotency is applied before the booking can be duplicated.

9.4 Release Semantics

Voucher quota is reserved at booking creation, represented by a **RESERVED** redemption, and counted in `used_count`. Confirmation transitions it to **CONSUMED**. Expiration/cancellation transitions only a **RESERVED** redemption to **RELEASED**, decrements the quota in the same transaction, and keeps the audit row. Repeating release affects zero rows, so the allocation cannot be decremented twice.

9.5 Operation Scope

The elevated API supports creating an active, time-bounded percentage or fixed voucher with optional concert/category scope. Update and delete endpoints are not implemented: changing a policy already referenced by historical or active redemptions requires explicit versioning/audit rules beyond this assessment.

9.6 Correctness Evidence

CONC-003 has 20 users compete for one remaining global voucher unit and verifies exactly one successful redemption and a quota that never exceeds its limit. Integration tests also prove eligibility/scope errors roll back ticket inventory and that release preserves the redemption audit row while decrementing `used_count` only once.

Booking Lifecycle and Reservation Expiry

10.1 State Model

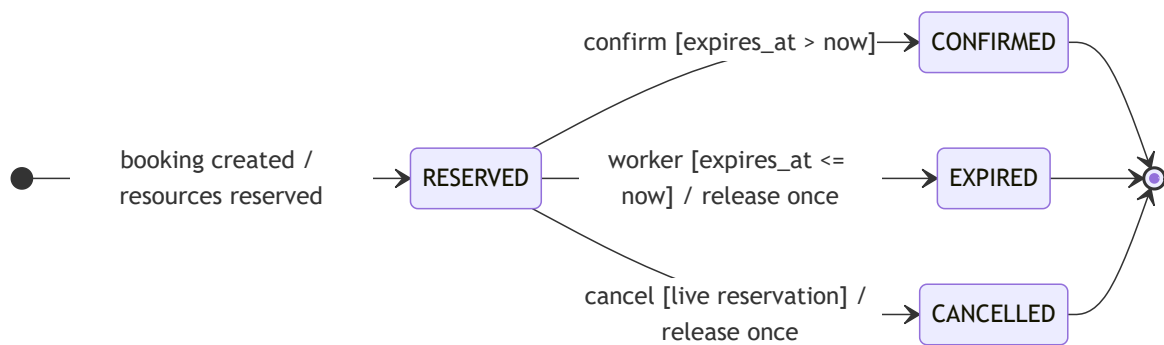


Figure 10.1: Implemented booking lifecycle

Every terminal update requires the current state **RESERVED**; exactly one competing confirm, cancel, or expiry transition can change the row.

Table 10.1: Allowed lifecycle transitions

From	To	Allowed	Guard / effect
RESERVED	CONFIRMED	Yes	Must still be live; resources remain consumed
RESERVED	CANCELLED	Yes	Customer requires live reservation; release resources once
RESERVED	EXPIRED	Yes	Must be past expiry; release resources once
CONFIRMED	Any state	No	Terminal in current scope
EXPIRED	Any state	No	Terminal in current scope
CANCELLED	Any state	No	Terminal in current scope

10.2 Why Reservation TTL Exists

Atomic creation prevents immediate overselling, but a reservation that never confirms must not hold finite resources forever. New bookings therefore store `status = RESERVED` and a configurable `expires_at` (ten minutes by default). TTL separates a short reservation transaction from later confirmation and bounds abandoned capacity.

10.3 Expiry Worker

The worker wakes every 30 seconds by default, prevents overlapping ticks in one process, requests a default batch of 100, and delegates one transactional batch to the lifecycle service. The service clamps externally supplied batch sizes to 1–500. Figure 10.2 shows the implemented flow.

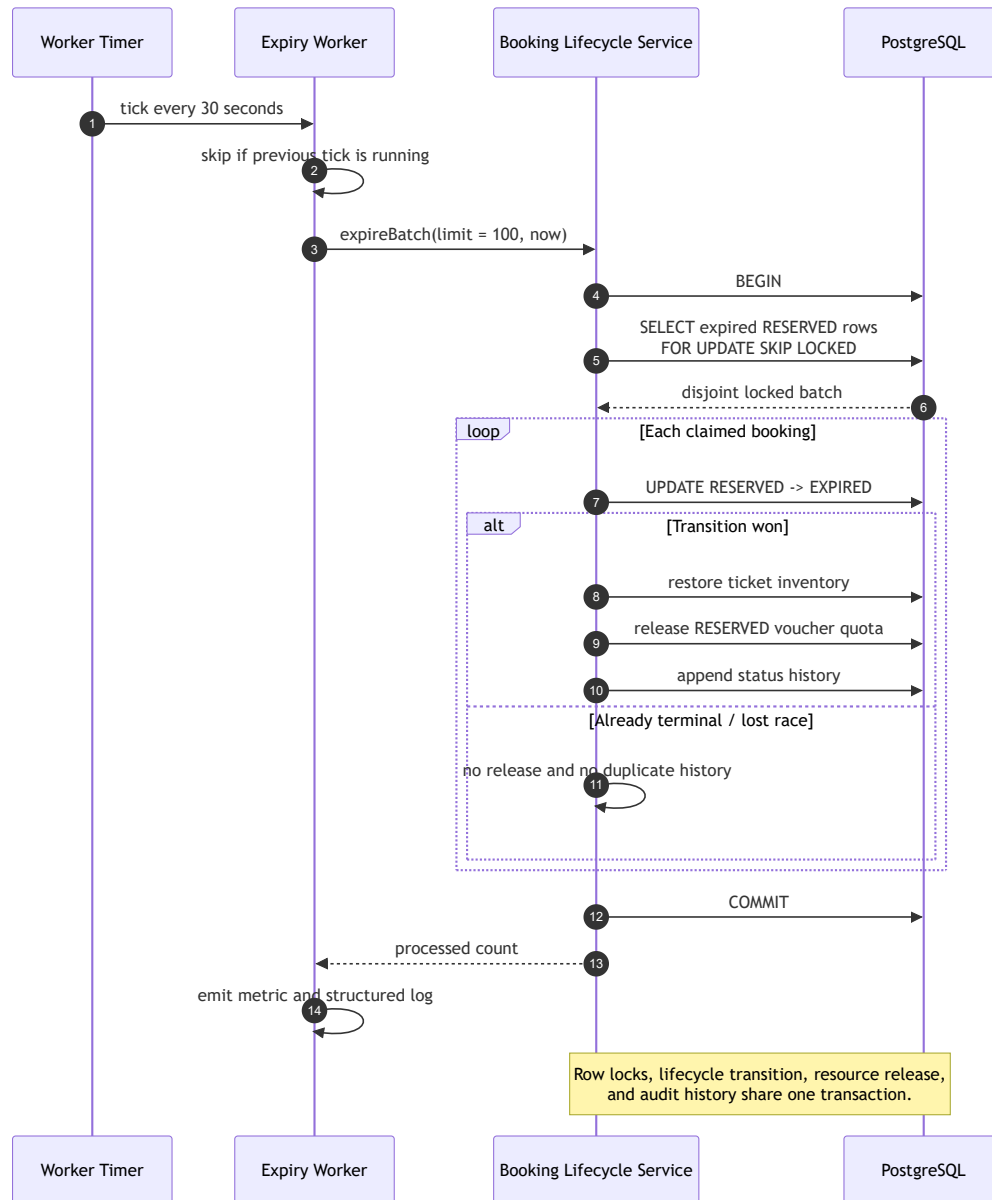


Figure 10.2: Reservation expiry worker sequence and transaction boundary

10.4 Scalable Worker Claiming

```

1 SELECT id
2 FROM bookings
3 WHERE status = 'RESERVED'
4   AND expires_at <= NOW()
5 ORDER BY expires_at, id
6 LIMIT :safe_limit
7 FOR UPDATE SKIP LOCKED;

```

Listing 10.1: Disjoint batch claim for expired reservations

Multiple workers claim disjoint rows without a distributed lock. Each claim lock is held until the conditional status update, ticket restoration, reserved voucher release, and audit insert commit. A repeat run cannot find the row as **RESERVED** and therefore cannot release resources twice.

10.5 Confirm-vs-Expire Race

```
1 UPDATE bookings
2 SET status = 'CONFIRMED', updated_at = NOW()
3 WHERE id = :booking_id
4     AND status = 'RESERVED'
5     AND expires_at > NOW();
```

Listing 10.2: Conditional confirmation

The expiry and confirmation paths both require **status = RESERVED**; the database row update is the arbitration point. The expiry/time predicate is evaluated after any lock wait, so stale application time cannot confirm an already-expired reservation. **CONC-004** proves one terminal outcome and a resource state consistent with that winner.

API and Operation Workflows

OpenAPI is the complete machine-readable contract and Postman contains the executable workflows. This chapter highlights only the endpoints that expose the design's important boundaries.

11.1 Representative Customer APIs

Table 11.1: Representative customer endpoints

Method	Endpoint	Purpose
GET	<code>/api/concerts</code>	Published catalog; detail/category routes extend this read model
POST	<code>/api/bookings</code>	Idempotent atomic reservation with optional voucher
GET	<code>/api/me/bookings</code>	Ownership-scoped booking history and detail
POST	<code>/api/bookings/{id}/confirm</code>	Guarded lifecycle command; cancel is the symmetric release command

11.2 Representative Operation APIs

Table 11.2: Representative operation endpoints

Method	Endpoint	Purpose
GET	<code>/api/admin/bookings</code>	Monitor/filter; detail includes audit and voucher state
PATCH	<code>/api/admin/bookings/{id}/status</code>	Guarded, reasoned, audited transition
POST	<code>/api/admin/concerts</code>	Draft creation; category and publish commands complete the minimal setup
POST	<code>/api/admin/vouchers</code>	Create validated global or scoped promotion

Operation routes require `OPERATOR` or `ADMIN`. Manual terminal commands start from `RESERVED`; cancellation/expiry release resources in the same transaction and every successful command appends status history. Update/delete administration is intentionally omitted because historical pricing and voucher-version rules are unspecified.

11.3 Availability Semantics

Read availability is an informational snapshot and can become stale immediately. The authoritative decision is the conditional inventory update inside `POST/api/bookings`.

11.4 Error Contract

```

1 {
2   "code": "INSUFFICIENT_TICKET_INVENTORY",
3   "message": "Requested ticket quantity is no longer available",
4   "details": { "ticketCategoryId": 10 },
5   "traceId": "..."
6 }
```

Listing 11.1: Representative business error

Expected sold-out, voucher, idempotency, ownership, and rate-limit conflicts produce stable documented 4xx responses rather than generic 500 errors.

Security, Reliability, and Observability

12.1 Security Boundary

Every customer or operation request requires a syntactically valid `x-user-id` that exists in the seeded `users` table. Customer routes require `CUSTOMER`; `/api/admin/**` requires `OPERATOR` or `ADMIN`. Booking reads and customer commands include the customer ID in the lookup predicate, so another customer's booking is not disclosed.

This header is assessment-grade identity injection, not authentication. A production deployment must derive the same subject/role from a verified JWT, OIDC provider, or trusted gateway. Existing ownership and RBAC checks remain necessary after that replacement.

Validation pipes reject malformed DTOs and non-whitelisted fields. The booking service rejects unpublished concerts, cross-concert categories, invalid quantities, untrusted price inputs, and invalid voucher policy. Configuration comes from environment variables; no runtime secret or private key is committed. Errors never expose stack traces, SQL, request hashes, or stored replay bodies.

12.2 Reliability Controls

Correctness comes from short ACID transactions, database constraints, conditional writes, bounded classified DB retries, and idempotency. Liveness is process-only; readiness checks PostgreSQL and returns `503` when the API cannot commit. Controlled sold-out/voucher/idempotency/rate-limit outcomes use stable `4xx` contracts rather than being counted as infrastructure failures.

The process-local booking-attempt limiter defaults to 60 attempts per customer per 60 seconds and returns `429` with `Retry-After`. It reduces one instance's abuse/retry pressure

but never replaces inventory or idempotency correctness. Multi-instance production requires a shared gateway/Redis policy.

12.3 Observability

The API validates, creates, and propagates `X-Request-ID`; responses and standard errors expose the same correlation value. One-line JSON logs include bounded request/event fields and omit request bodies, access tokens, voucher codes, and payment data.

`GET /metrics` exposes low-cardinality Prometheus text counters including HTTP totals, booking attempts/successes/replays/failures, sold-out and inventory conflicts, voucher exhaustion, idempotency conflicts, and expired bookings. These counters are in-process and reset on restart.

12.4 Implementation Status and Evolution

Table 12.1: Security, reliability, and observability status

Capability	Status	Boundary / evolution
Ownership and operation RBAC	Implemented	Backed by seeded user roles and scoped queries
Signed authentication / identity provider	Future	Replace trusted header with verified claims
DTO/business validation and stable errors	Implemented	Central validation and sanitized exception filter
Request correlation and structured logs	Implemented	Local JSON output; forward to managed log storage later
Liveness/readiness and metrics endpoint	Implemented	Metrics storage is currently single-instance memory
Booking rate limiter	Implemented locally	Move key/state to gateway or Redis for multiple replicas
Distributed tracing and alerting	Designed, not implemented	Export OpenTelemetry/Prometheus and define SLO alerts
Secret manager and production TLS/WAF	Future	Deployment/platform responsibility outside submission

Testing Strategy and Correctness Evidence

13.1 Testing Layers

Table 13.1: Testing strategy

Layer	Purpose	Examples
Unit	Pure domain policies	Pricing, voucher eligibility, state machine, request fingerprint
Integration	Real PostgreSQL semantics	Transactions, constraints, repository SQL, rollback behavior
Concurrency	Prove critical invariants under races	Oversell, idempotency race, final voucher, confirm-vs-expire
API/Postman	Reviewer-visible workflow validation	Customer happy path, admin operations, documented edge cases
Load	Measure behavior under stated and burst traffic	Throughput, latency, expected 4xx vs system errors

13.2 Invariant-to-Test Evidence

Table 13.2: Critical correctness evidence

Test ID	Invariant	Scenario / Expected Result
CONC-001	INV-01	100 requests compete for 10 tickets; exactly 10 succeed; final stock=0
CONC-002	INV-02	20 concurrent requests use same user/key/payload; one logical booking exists
CONC-003	INV-03	20 users compete for final voucher quota=1; exactly one redemption succeeds
CONC-004	INV-05	Confirm and expiry race; exactly one valid final outcome and resource state
INT-001	INV-04	Voucher reservation failure causes ticket reservation to roll back
INT-002	INV-06	Changing current ticket price does not change existing booking total

13.3 Requirement Traceability

The traceability chain is **Requirement** → **Invariant** → **Design** → **Implementation** → **Test Evidence**. Representative critical paths are listed below; OpenAPI, Postman, and the automated suites remain the complete executable inventories.

Table 13.3: Critical requirement traceability

Req.	Invariant	Design	Implementation evidence	Test evidence
FR-05, NFR-01	INV-01	Conditional quantity update	<code>inventory-reservation.repository.ts</code> + DB checks	CONC-001
FR-20, NFR-02	INV-02	User/key/fingerprint claim and replay	<code>create-booking.service.ts</code> , <code>idempotency.repository.ts</code>	CONC-002
FR-06, NFR-03	INV-03	Conditional quota + active per-user uniqueness	<code>voucher-reservation.repository.ts</code> , migration 0004	CONC-003
FR-05, NFR-04	INV-04	One booking transaction	<code>create-booking.service.ts</code> , transaction helper	INT-001
FR-17-19	INV-05, INV-07	Conditional terminal transition + locked batch	lifecycle service/repository + expiry worker	CONC-004
FR-03, FR-05	INV-06	Server pricing and immutable snapshots	booking write repository + DB schema	INT-002
FR-11	INV-07, INV-08	Guarded transition and status history	operation service + lifecycle repository	Operation integration suite

13.4 Evidence Standard

Concurrency tests should verify both the HTTP/API result and final database state. A test that only counts response codes without checking inventory, booking rows, and voucher rows is insufficient evidence for consistency guarantees.

Performance and Flash-Sale Analysis

14.1 Expected Workload and Evaluation Goal

The stated peak of 300–500 booking requests per minute is approximately 5–8.3 requests per second. The evaluation therefore targets stable behavior at that rate and a short burst above it. It does not claim a universal capacity limit: flash-sale risk is dominated by hot-row contention, transaction length, and controlled conflict behavior rather than aggregate request volume alone.

14.2 Executed Environment

Table 14.1: Reproducible local load-test environment

Field	Value
Benchmark source commit	<code>cfc291ac1ad3e6ac9ebb09d304db60cead07a62e</code>
Host	Windows x64; Intel Core i5-12450H (8 cores / 12 logical processors); 15.6 GiB RAM
Runtime	Node.js v22.17.1; one API container
Database	PostgreSQL 16.14, one local Docker container
Load engine	<code>grafana/k6:0.53.0</code> through Docker
Target	<code>http://host.docker.internal:3300;</code> <code>RATE_LIMIT_BOOKING_MAX=1000</code> confirmed in API container
Dataset	Deterministic customer, concert <code>summer-festival-2026</code> , STANDARD inventory

14.3 Executed Scenarios

The k6 script ran 5 iterations/second for 30 seconds, followed by a three-stage burst that ramped to 15 iterations/second and back over 30 seconds. It generated unique idempotency keys so each iteration represented a new logical booking. The rate limit was raised only to prevent the resilience guard from obscuring booking/database behavior.

Hot inventory, final-voucher, sold-out, and lifecycle contention are covered by dedicated deterministic concurrency suites. They were not represented as separate latency/capacity scenarios in this k6 baseline; presenting them as measured load results would overstate the evidence.

14.4 Measured Results

Evidence Verified k6 baseline

Requests: 501 completed / 0 interrupted

Throughput: 8.35 requests/second (reported as 8 req/s in the committed summary)

Latency: p50 13.31 ms; p95 24.87 ms; p99 31.73 ms; maximum 47.63 ms

Booking outcomes: 501 successes; 0 controlled business rejects

System error rate: 0.000%

Thresholds: system error < 1%, p95 < 1000 ms, and p99 < 2000 ms all passed

The exact generation time, database build string, reproduction command, and summary are committed in `docs/PERFORMANCE_REPORT.md`. Controlled 400/409/429 outcomes are intentionally classified as business rejects, not server failures. This particular run had none because the seeded STANDARD capacity and limiter were sufficient for all 501 unique bookings.

14.5 Interpretation and Limitations

The measured baseline is comfortably inside the stated workload and configured thresholds on one local API/database pair. It demonstrates absence of 5xx or unexpected status under this profile, not production capacity, multi-instance behavior, or saturation. CPU allocation to Docker, database locality, connection pool limits, and the non-hot STANDARD category materially influence the result.

Before a production claim, execute longer steady-state and soak tests; isolate sold-out and single-hot-category latency; measure final-voucher contention; capture database connection/lock wait/CPU/I/O; and repeat with the intended deployment topology and shared rate limiter. Optimization should preserve short transactions and deterministic resource order; admission control, partitioning, or queueing needs measured hot-row evidence.

Scope, Limitations, and Future Evolution

15.1 Implemented and Omitted Scope

Table 15.1: Reviewer-facing implementation scope

Capability	Status	Boundary
Concert browse and availability	Implemented	Published concerts; availability is informational and rechecked during booking
Atomic booking and retry safety	Implemented	Server prices, inventory, optional voucher, persistence, and idempotency share one transaction
Voucher create and redemption	Implemented	Percentage/fixed, time, quota, and optional concert/category scope
Voucher update/delete	Not implemented	Historical and versioning semantics are intentionally omitted
Reservation lifecycle and expiry	Implemented	Guarded terminal transitions and exactly-once release
Operation workflows	Implemented	Monitor bookings, guarded status change, concert/category create and publish
Real payment processing	Out of scope	Confirmation is simulated; no provider or financial settlement
Seat-level allocation	Out of scope	Inventory is category-level capacity
Notifications and asynchronous events	Not implemented	Future post-commit integration
Multi-region and distributed controls	Out of scope	PostgreSQL is the single consistency boundary; limiter and metrics are process-local

15.2 Known Limitations

The current identity header is an assessment mechanism rather than production authentication. Confirmation does not prove payment, inventory is not seat-specific, and the process-local limiter/metrics need shared infrastructure for multiple API replicas. The polling expiry worker provides bounded rather than instantaneous expiry. Detailed assumptions remain in `docs/ASSUMPTIONS_SCOPE_LIMITATIONS.md`.

These limitations do not weaken the submitted guarantees: inventory cannot oversell, one idempotent intent cannot double-book, voucher quota cannot over-redeem, and competing terminal transitions cannot release resources twice.

15.3 Payment Processing Scope

Real payment processing is outside the assessment scope. The implementation provides payment-adjacent behavior: a time-limited **RESERVED** booking, immutable totals, conditional **RESERVED** → **CONFIRMED**, cancellation or expiry, exactly-once resource release, and retry-safe booking creation.

There is no provider client, payment session, authorization/capture, webhook verification, payment-state table, settlement, refund, chargeback, or reconciliation. A future flow is deliberately separated from the current transaction:

Reserve + commit **RESERVED** → External gateway → Verified webhook → New
transaction → Conditional confirm

Decision External-call boundary

Never keep the booking/inventory transaction open while calling an external payment provider. Provider interaction occurs after commit; its verified result is handled idempotently in a new transaction and cannot overwrite an already expired or cancelled booking.

15.4 Decision Summary and Revisit Triggers

Full ADRs remain under docs/adr/; the report keeps only the triggers that could invalidate today's choices.

Table 15.2: Architecture revisit triggers

Decision	Revisit when
Modular monolith	A module needs independent ownership/deployment/scaling and its consistency boundary can be separated
PostgreSQL atomic inventory	Measured hot-row waits or write-region limits violate the target SLO
Database idempotency	Retention volume or cross-region replay requires a dedicated lifecycle/storage strategy
Polling expiry worker	Poll delay, scan cost, or worker volume justifies broker scheduling
Outbox remains optional	A real payment, notification, or analytics consumer requires guaranteed post-commit publication

Conclusion

The assessment backend is designed around explicit, traceable guarantees rather than a large number of loosely specified features. PostgreSQL provides the consistency boundary for ticket inventory, voucher quota, booking state, audit history, and idempotency. The modular monolith keeps the critical booking transaction understandable and testable, while scalable row claiming and the optional transactional outbox provide credible evolution paths without being prerequisites for current correctness.

The report connects requirements to business rules, invariants, design choices, implementation evidence, and automated tests. Concurrency races, rollback paths, idempotent retries, lifecycle arbitration, CI smoke, and the measured k6 baseline provide reproducible evidence rather than prospective claims. OpenAPI, Postman, migrations, seed data, runbooks, and ADRs support independent reviewer verification.

The system's limitations are equally explicit. It does not process payments, select seats, send notifications, administer full catalog/voucher lifecycles, or claim multi-region production readiness. Those boundaries preserve an honest assessment result: the implemented workflows are internally consistent and defensible, while future payment and asynchronous integrations have a safe post-commit path that does not weaken the finite-resource transaction.